

SPD：潜边轮廓图

定义与证明资料 / 中文版

定义，固定版 v0.1，2026-09-14。NER 展开器 · Python 定义 · 标准计数解码器 · 良序论文。

SPD = **Slot Profile Diagrams**。表达式是有限列图，每条关系只有四个自然数；头和普通参数由被引用列的关系递归读出，不另存树、层号、序数秩或操作历史。共享 DAG 只是比较时重建的索引，不是记号的第五个字段。

本页固定最高可见头纤维、LATENT 低包、严格连接守卫和后块同头携带这套规则。其他实验性替换或合并规则不属于 SPD v0.1。软件预算也不是数学规则。

现有良序结论为纸面论证，尚无 SPD Lean 证书。全部合法图的展开良基与标准可达域的列序良序是不同结论；SPD 与 ARD、ARD2、wY、IPD 的**整个序型**大小关系均未证明。

1. 有限列语法

有限图为 $G = (C_0, \dots, C_{m-1})$ 。第 j 列是有限关系集，每条关系为

$$(h, p, s, q), \quad 0 \leq h, s \leq p < j, \quad q \in \{0, \dots, p\} \cup \{j, j+1\}.$$

h 是头来源列， p 是父列， s 是普通参数来源列；子列就是所在列 j 。根状态 q 如下：

状态	整数编码	列表显示
ROOT	$0 \leq q \leq p$	数字 q
SELF	$q = j$	*
LATENT, 待激活	$q = j + 1$	+

LATENT 不是对下一列的引用；以后追加第 $j+1$ 列不会改变它的意义。同一 (h, p, s) 只保留最大的 q ，重复关系没有影响。较小 q 不作为另一条可见行同时读入头。

无损列表每列用一对方括号，关系以分号分隔，四个坐标以逗号分隔。例如

```
[][0,0,0,*][1,1,1,*]
```

空列为 $[]$ ，无列的零图为 $\emptyset$ 。自然数 m 表示 m 个空列；它不是“计数为 m 的复杂列”。输入或构造时先按上述规则规范化。

2. 从列关系读出的递归头

2.1 辅助项序

辅助有限项由 Z 、地址原子 $\text{atom}(r)$ 、两个形式字母 A, B 及

$$N(h; t_0, \dots, t_{k-1})$$

构成。头 h 和每个普通参数 t_i 都是同一语法的项， $k \geq 0$ 。形式常量顺序为

$$Z < \text{atom}(0) < \text{atom}(1) < \dots < A < B,$$

全部常量小于全部 N 项。两个 N 项 a, b 按以下顺序比较：

1. 若 a 的某个直接普通参数不小于 b ，则 $a > b$ 。
2. 否则，若 b 的某个直接普通参数不小于 a ，则 $a < b$ 。
3. 否则先比头，再比参数个数，最后逐项比较等长参数列表。

这是头优先级为“头、元数”的具体字典路径序 LPO。头本身不混入前两步的普通参数列表。递归进入更小的有限项，不运行基本列；元数优先级不可省略。没有大于所有复合项的全局 CAP。

A 表示未来父， B 表示未来端点。代入 $A = p$ 的意思是以 $\text{atom}(p)$ 替换所有深度的 A ； B 仍为公共最大原子，不是一个比所有复合项大的顶端。

2.2 第 j 列的头 H_j

按列从左向右计算。先忽略 LATENT 行；没有可见行时置 $H_j = Z$ 。否则在可见行中取形式键 (H_h, h) 最大的唯一头索引 h_* ，只读取 $h = h_*$ 的行，按数字地址 (p, s, q) 递减排列。每行贡献参数对

$$(H_s, Q), \quad Q = \begin{cases} A, & q = p, \\ B, & q = j, \\ \text{atom}(q), & q < p. \end{cases}$$

把这些参数对依次串接，定义 $H_j = N(H_{h_*}; H_{s_1}, Q_1, \dots, H_{s_r}, Q_r)$ 。此处 $r \geq 1$ 。较低头的可见行及全部 LATENT 行仍参与列比较和展开，只是暂不属于当前派生头的最高可见纤维。

引用 H_h, H_s 时，它们内部的 A, B 保持为形式变量，不先代入历史列的父或子地址。所有固定 ROOT 都严格早于 j ；将来在父 $p \geq j$ 使用 H_j 时，形式 ROOT 小于 $A = p < B$ ，所以最高头纤维的选择在合法严格地址重标下不变。

实现可对相同子表达式共享存储，并用显式栈比较；无需展开重复子树。派生索引不随输入保存为冻结头。

证明中的派生深度为：常量深度 0， $d(N(h; t_0, \dots, t_{k-1})) = \max(d(h) + 1, d(t_0), \dots, d(t_{k-1}))$ 。深度初段引理给出 $d(a) < d(b) \Rightarrow a < b$ ，所以 Python/JavaScript 可先用缓存深度区分项。这是上述 LPO 的等快捷径，不是新的比较规则或另一个输入层号。

3. 直接按列比较

第 j 列关系 $e = (h, p, s, q)$ 的剖面为

$$P(e) = (H_h[A = p], h, H_s[A = p], s, q).$$

B 在所有被比较剖面中是同一公共最大端点原子； q 按实际整数编码比较，等价于 ROOT 顺序后接 SELF、LATENT。

每列按 $(p, P(e))$ 递减排列，并按该键逐关系字典比较；真前缀较小。图从左向右比较列，整式真前缀也较小。第一不同列以前的前缀相同，所以所用引用的头已经共同确定。比较不搜索展开路径。

控制行使用不同优先级：在末列按 $(P(e), p)$ 取最大者。不同父的 $P(e)$ 必须分别代入各自的 p ，不能无条件在一个统一父上下文比较形式 H 。

4. 基本列的完整有限规则

指标 n 为非负整数， $0[n] = 0$ 。对非空有限图令 $x = m - 1$ ，以 ∂G 表示删末列。规定

$$G[0] = \partial G, \quad G[n] = \partial G \quad \text{若 } C_x = \emptyset.$$

其余情形 $n > 0$ 且末列非空。固定旧控制 $e = (h, c, s, q)$ ，令 $L = x - c > 0$ 。先保留 $C_0, \dots, C_{x-1}$ 。之后顺序追加 n 个块；每块是“降低接缝、非空时才有的连接列、独立纯来源副本”。

4.1 搬运及块地址

第 b 块前累计位移为 d_b ， $d_0 = 0$ ，并置

$$\phi_b(i) = \begin{cases} i, & i < c, \\ i + d_b, & i \geq c, \end{cases} \quad S_b = x + d_b, \quad c_b = \phi_b(c).$$

搬运第 j 列的一条关系到子列 j' 时，前三坐标按地址映射搬运，根按原来的角色解码：

$$q' = \begin{cases} j' + 1, & q = j + 1, \\ j', & q = j, \\ \phi(q), & q \leq p. \end{cases}$$

因此一般严格映射下 LATENT 变成 $\phi(j) + 1$ ，不是 $\phi(j + 1)$ 。先搬运，再在目标地址中降低。

4.2 降低接缝 S_b

搬运旧末列全部非控制行。记搬运后的控制为 (h_b, c_b, s_b, q_b) ；仅把它的根降一级：

$$\text{LATENT} \longrightarrow \text{SELF} \longrightarrow c_b \longrightarrow c_b - 1 \longrightarrow \cdots \longrightarrow 0 \longrightarrow \text{删掉该行}.$$

随后对全部 $0 \leq k, t \leq c_b$ ，若

$$(H_k[A = c_b], k, H_t[A = c_b], t) < (H_{h_b}[A = c_b], h_b, H_{s_b}[A = c_b], s_b),$$

则加入 $(k, c_b, t, \text{LATENT}_{S_b})$ 。

仅当 $b \geq 1$ ，还从**原输入**的切点列 C_c 携带全部 LATENT 行以及头索引等于原控制 h 的可见行。其余可见行不携带。坐标按 ϕ_b 移动，来源列自己的 SELF/LATENT 改成 S_b 的相应标记。第一块不携带来源列。对合并后的关系规范化，追加 S_b 。

同头携带保留可继续读取的原参数纤维，换头时不携带旧可见纤维；所有待激活槽位仍保留。

4.3 严格连接守卫

接缝已加入后，用旧引用列的模板计算

$$\hat{P}_{S_b} = (H_{h_b}[A = S_b], \uparrow h_b, H_{s_b}[A = S_b], \uparrow s_b), \quad \uparrow i = \begin{cases} S_b, & i = c_b, \\ i, & i \neq c_b. \end{cases}$$

这里只提升地址坐标，**不把旧模板换成** H_{S_b} 。例如 $h_b = c_b$ 时，第一项仍是旧切点列的头在 $A = S_b$ 的求值，第二项才改为 S_b 。

遍历 $0 \leq k, t \leq S_b$ 。当且仅当

$$(H_k[A = S_b], k, H_t[A = S_b], t) < \hat{P}_{S_b},$$

向连接列加入 $(k, S_b, t, \text{LATENT})$ 。该列位于 $S_b + 1$ ，其 LATENT 整数为 $S_b + 2$ 。若关系集非空才追加连接列；置 $\varepsilon_b = 1$ ，否则置 $\varepsilon_b = 0$ 。

守卫是严格不等式；相等四坐标不能通过。 k 或 t 可以等于新接缝 S_b ，所以连接关系能够真正使用新造列，而不仅引用固定旧头库。

4.4 独立纯来源

更新

$$d_{b+1} = d_b + L + 1 + \varepsilon_b.$$

按 ϕ_{b+1} 完整复制原 $C_c, \dots, C_{x-1}$ ；其第 j 列放在 $\phi_{b+1}(j)$ 。这份副本不受同头携带筛选，也不与接缝合并。所有 SELF/LATENT 按复制列自身的角色重绑。

这样下一块重新拥有旧来源头的精确搬运。一次展开只有上述有限循环和有限头比较，不递归展开新生成的图，也不靠“先运行到零”定义基本列。

4.5 直接结果及例子

每块宽为 $L + 1 + \varepsilon_b$ ，它只依赖已形成的前缀，不依赖最后总指标 n 。因此 $G[n]$ 是 $G[n + 1]$ 的完整列前缀；旧末列非空时为真前缀。旧末列以前的列全部不变，首次差异是第一接缝的严格降低。因此每个 nonzero 有限图的基本项都严格小于原图； $0[n] = 0$ 不算严格下降边。

```
S1    = [][0,0,0,*]
S1[1] = [][0,0,0,0][0,1,1,+;0,1,0,+][[]]
counts: 1,3 -> 1,2,9,1
```

5. 种子、标准域和良序范围

种子 S_n 有 $n + 1$ 列：

$$C_0 = \emptyset, \quad C_j = \{(j-1, j-1, j-1, j)\} \quad (1 \leq j \leq n).$$

外加最大顶端 Top，显示为 Limit of SPD，定义 $\text{Top}[n] = S_n$ 。特别地 $S_0 = []$ ，不是零； $S_{n+1}[0] = S_n$ ， $S_0[0] = 0$ 。

令 $D(H)$ 包含 H 及所有有限展开后代，忽略零自环。有限标准域是 $\text{Std} = \bigcup_{n < \omega} D(S_n)$ ，完整记号再加顶端。标准性是有限种子可达性，不是辅助语义表示的存在性。结构解析器本身不证明手写列表标准；第 7 节给另外的成员判定方法。

良序论文的纸面公理上界为 $KP_\omega + \text{“存在不可数序数”}$ ，含完整集合归纳，不要求幂集或选择公理。论文先给辅助头实际集合秩，再对有限需求关系加入严格较低的新父请求，以可数见证操作闭包证明供应，最后逐块降低表示的末标签。它不假设目标良序或供应器。

结论分为：

- 所有结构合法有限图的非零展开关系良基。
- 指定首差列序在 Std 上良序，加入最大顶端后仍良序。

第二项利用前缀基本列：较小指标的基本项可由较大指标反复删末列到达；对下降秩归纳，每个种子后代锥按可达性可比。种子互为前缀，任意两个标准式有共同种子祖先。因此标准列序与真后代关系一致。

全部合法 raw 图的列序却不良序。例如令 R_j 有 j 个空列，随后一列为 $(0, j-1, 0, 0)$ ， $j \geq 1$ ，则 $R_j > R_{j+1} > \dots$ 。这是列序降链，不是展开链。不能删去上述标准域限制。

这是纸面证明状态，不是 SPD 的 Lean 形式化或对 Python/JavaScript 虚拟机的证明。局部头库可以在实际展开中再生和重挂，但局部轮廓的表达力不自动给出完整图序的大小。当前不宣称 SPD 与 ARD、ARD2、wY、IPD 的序型不等式，也不声称公理上界最优。

6. 精确局部计数

固定某列之前的前缀，将该列视作末列。作正指标展开后删去新追加尾部，只保留第一接缝；重复至该列清空，再删除空列。局部计数包括最后删除，所以空列计数为 1。第一块无来源携带，保留的局部步骤与正指标无关。

对位置 j 的每个实际出现的父 p ，把 $i = 0, \dots, p$ 按 $(H_i[A = p], i)$ 排序，令 $r_p(i)$ 为零起始秩。取该父下剖面最高的关系 (h, p, s, q) ，定义

$$\delta_p(q) = \begin{cases} q + 1, & q \leq p, \\ p + 2, & q = j, \\ p + 3, & q = j + 1. \end{cases}$$

精确公式是

$$\text{count}(C_j) = 1 + \sum_p [\delta_p(q) + (p + 3)((p + 1)r_p(h) + r_p(s))].$$

每个父独立进行根数位的减一和较低 pair 的 LATENT 补齐；因此每次局部步骤使总式恰减一。较低 pair 原先的内容不影响这个总剩余次数。程序直接计算公式，不逐步清空来计数。

所有合法第 j 列都满足

$$\text{count}(C_j) \leq B(j) = 1 + \sum_{p < j} (p+3)(p+1)^2 = 1 + \left(\frac{j(j+1)}{2} \right)^2 + \frac{j(j+1)(2j+1)}{3}.$$

这只是列位置的四次界，不是完整降零路径长度或一次展开列数的界。种子计数依次为 1, 3, 16, 45, 96, 175, ...；第 $j \geq 1$ 个种子列计数为 $j^2(j+2)$ 。计数不据此与别的记号中同样的数字串等同。

7. 标准计数的有限反解

每次真实展开都严格降低计数词的字典序：正指标的首次差异减一，删列给真前缀。标准后代锥与列序一致，故计数在 Std 上严格保序且单射；raw 图之间仍可能碰撞。

7.1 两个有限宽度事实

固定宽度界 $m > 0$ 。 S_{m-1} 是宽不超过 m 的最大标准图：更宽种子的前 m 列就是它；任何宽不超过 m 且不大于该宽种子的图，也不能首次超过这个前缀。

对非空标准 G ，宽不超过 m 时，置

$$T_m(G) = \text{prefix}_m(G[m]).$$

它是宽不超过 m 的最大真标准后代。截前缀是有限次 $[0]$ ；末列非空时， $G[m]$ 已至少有 m 列，而其他基本项由前缀性与它比较。任何宽不超过 m 的真后代位于某个基本项之下，因而也在这个前 m 列之下。末列为空时， T_m 就是删末列。

7.2 算法与正确性

输入正整数词 $(t_0, \dots, t_{m-1})$ ；空词返回零。非空时从 $G = S_{m-1}$ 开始，依次处理 $j = 0, \dots, m-1$ ：

1. 若缺少第 j 列，或其当前计数 $c < t_j$ ，拒绝；若 $c = t_j$ ，继续下一位。
2. 若 $c > t_j$ ，截到 $j+1$ 列，再作 $c - t_j - 1$ 次固定前缀局部步骤，使该位变为 $t_j + 1$ 。
3. 取 $G := T_m(G)$ ，该位恰再减一成为 t_j ，已匹配的前位不变。
4. 全部位完成，返回规范列图 G 。

若存在目标标准图 A ，算法保持 $A \leq G$ 。截尾和局部下降在首次不同位仍严格高于目标；最后取最大有限宽真后代也不越过 A 。所以有效词不被拒绝。反之，成功返回的是实际标准后代且计数精确匹配；由单射性它唯一。宽度不足或偏小因而是已证明的无效输入，而非未找到路径。

最多 m 个修正阶段，局部下降总数不超过 $\sum_{j < m} B(j) = O(m^5)$ ，另有至多 m 次 T_m 。这不是未经分析的总执行时间或内存复杂度。实现可用任何足以达到 m 列的较小正指标代替 $[m]$ ：当前末位置为 j 、控制父为 c 时，每块至少 $j - c + 1$ 列，故指标 $\max(1, \lceil (m-j)/(j-c+1) \rceil)$ 已足够，前 m 列不变。

NER 支持 1, 3, 16 或 $C(1, 3, 16)$ ，均解为 S_2 ； $C()$ 为零， $C(1, 1, 1)$ 为三个空列。单个自然数仍是空列数。 $C(1, 4)$ 非法，尽管存在计数为 $(1, 4)$ 的 raw 两列图。

Python 解码器提供 `decode_counts`。`InvalidCounts` 表示不存在对应标准式或输入不是正整数词；`TimeoutError`、`OverflowError` 仅表示预算不足，不能当作非法、假阴性或数学不终止。默认宽度上限 32、局部步骤上限 100000，整个调用共享同一个 `spd.Budget`，不在内部反复重置时间。

一般 raw 图的标准性也可判定：规范化并计算计数，解出唯一标准代表，再比较完整列图。**只检查计数可解不够。**

`is_standard` 实现此方法；非法结构仍抛 `ValueError`，资源异常不被转成 `False`。

8. 软件使用与资源保护

8.1 NER

在 NER 自定义记号入口导入完整 SPD.ne-rewritten.js，选择 **SPD**。它没有外部依赖或网络请求，注册 ID 为 spd-research-20260914。默认显示列表，等价表示仅增加计数序列，不重复列表按钮。

输入支持 S0、S3[2]、Top[3][2]、Limit of SPD、自然数、完整列列表和上述标准计数格式。忽略空白；展开路径算完后保存真实规范列，不保存路径作为表达式。FS、FS_alter、FS_short 完全同规，不偏移指标。顶端是极限；有限非零式恰在未列非空时被 is_limit 判作极限，未列为空时为后继。

列表和计数继承页面字体。计数用 BigInt。默认一般计算预算约 900 毫秒、计数显示另约 200 毫秒；还限制工作量、列、关系、共享节点、参数槽、比较与父上下文缓存。当前主要几何上限为 2048 列、60000 条关系、80000 个共享节点。全部常量见源码 LIMITS。

结构计算超限抛出 SPDLimit，不返回截断基本项。计数显示预算不足时显示“计数预算内未算完”，不是近似计数。数学无效计数与执行超限分开报告。

8.2 Python 模块与直接运行

将同目录文件置于模块搜索路径，使用 Python 3.10+：

```
import spd
from spd_count_decode import decode_counts, is_standard

a = spd.seed(2) # S2, 三列
assert spd.counts(a) == (1, 3, 16)
assert spd.fs(a, 0) == a[:-1]
assert spd.compare(spd.fs(a, 1), a) == -1
assert spd.fs(a, 1) == spd.fs(a, 2)[:len(spd.fs(a, 1))]
assert decode_counts((1, 3, 16)) == a
assert is_standard(a)
```

图是列与四元组组成的 list/tuple，规范结果为 tuple；零为 ()。seed、normalize、fs、compare、counts 是函数，不是 SPD 类的方法。compare 返回 -1, 0, 1；read 构造临时派生索引。主模块只处理有限图，不提供顶端对象或 NER 文字解析器。

各图操作先检查和规范化输入；负数、布尔值及非整数坐标或指标被拒绝。默认 Budget 为 15 秒、2000000 事件，并有节点、参数槽、比较对、父上下文、列数及累计关系分配限制。可把同一个预算传给多个调用；将某个限制显式设为 None 才禁用该项。默认保护不是无限计算许可，资源耗尽抛异常且不返回部分结果。

从仓库根目录直接运行：

```
python -B notations/SPD/spd.py
python -B notations/SPD/spd_count_decode.py
```

当前这两个入口只运行源码末尾的演示，分别打印种子计数与一次展开、以及 (1, 3, 16) 的解码结果；没有解析表达式或接收展开指标的命令行参数接口。编程调用使用上面的函数 API。独立实现及有限测试用于核对规则，不替代良序论文，也不构成 SPD 的 Lean 验收。